Our Information Retrieval Project

Team Briefing

1. What are we actually building?

We are going to build a **search engine that takes a user's query and retrieves the most relevant documents from a collection of documents**.

For example, imagine we have a collection of 1,000 research papers. A user searches:

> *"How does air pollution affect respiratory health?"*

Our search engine should find the papers that are most useful for that query and display them in the correct order, with the most relevant papers at the top.

But our project is not just about building a search bar. The main goal is to **implement and compare different search and ranking methods, understand where each method works or fails, and determine which method performs best for our dataset**.

2. Why is this an Information Retrieval project?

Our course teaches us how search engines work internally.

When someone searches for something, the search engine needs to process documents, identify matching content, calculate relevance scores, rank documents, and return results efficiently.

There are multiple ways to do this. Some methods depend mainly on matching words, while other methods try to understand the meaning of the query and the documents.

Our project will put the concepts from our lectures into practice and compare their actual performance.

3. What will we build, step by step?

Step 1 — Select a document collection

We need a dataset containing documents that our search engine can retrieve. We can start with Cranfield, a small scientific-document retrieval benchmark that is also used in our tutorial code. It is a convenient starting point because it includes search queries and relevance judgments.

Step 2 — Process the documents

We will clean and tokenize the text and experiment with techniques such as stop-word removal, stemming and lemmatization. We need to check whether these steps actually improve retrieval.

Step 3 — Implement different retrieval methods

We will begin with simple methods and gradually move to more advanced ones:

- **Boolean retrieval:** finds documents that satisfy specified search conditions.
- **TF-IDF with the Vector Space Model:** ranks documents using weighted word overlap.

- **BM25:** a stronger classical ranking method that considers term importance, term frequency and document length.
- **Query-Likelihood Language Models:** rank documents according to how likely they are to generate the query.
- **Dense retrieval:** represents queries and documents as embeddings to match meaning, even when different words are used.
- **Hybrid retrieval:** combines lexical and semantic retrieval to see whether the combination improves results.

Step 4 — Evaluate the methods fairly

We will give the same queries to each method and compare the results using measures such as Precision, Recall, MAP, MRR and NDCG.

We cannot simply look at the results and decide which method looks best. We need relevance judgments and evaluation measures to support our conclusions.

Step 5 — Investigate failures

We will examine cases where a method performs poorly.

For example, BM25 might work well when a query contains exact technical terms but miss a relevant document that uses different wording. Dense retrieval might identify similar meaning but miss an exact technical detail.

We need to investigate these cases and determine whether combining methods or adding techniques such as query expansion or re-ranking helps.

Step 6 — Build the final search interface

Once our core retrieval and evaluation code works, we will create a simple interface where users can enter a query, see ranked documents and potentially compare retrieval methods.

4. What makes this project more than a basic search engine?

The important part is the experimental comparison.

We are not going to claim beforehand that one method is the best. We will test each method on the same dataset and queries, measure retrieval quality and efficiency, and explain the results.

Our final conclusions should answer questions such as:

- Which method retrieves the most relevant documents?
- Which method puts relevant documents near the top?
- Which method is fastest?
- Does semantic retrieval find documents that keyword-based methods miss?
- Does combining BM25 and dense retrieval actually improve performance?
- Which preprocessing techniques help, and which do not?
- What are the limitations of the best-performing method?

We can also add a more advanced feature from our syllabus, such as relevance feedback, query expansion, diversity-aware re-ranking or near-duplicate detection, once the main system is working.

5. Which course topics will we use?

Our core implementation will use text processing, Boolean retrieval, TF-IDF, BM25, statistical language models, evaluation measures, and dense or hybrid retrieval.

Other topics, such as PageRank, HITS, topic models, index compression, advanced query-processing algorithms, learning to rank and RAG, can be considered as extensions if they fit our dataset and available time. We do not need to force every topic into the main system.

6. What is our initial plan?

We will start with the Cranfield dataset and the tutorial code, build a working classical search engine, and make sure our evaluation measures are correct. Then we will add dense retrieval and hybrid retrieval, compare the results, analyze failures, and build the interface around the final system.

The exact winning method is something we will discover from our experiments, not decide in advance.

> **In one sentence:** We are building and experimentally evaluating a search engine that compares classical and semantic retrieval methods to determine which approaches retrieve the most relevant documents, why they work, and where they fail.

How I Would Divide the Work Among the Team

Work areas for the team

For a team project, these are reasonable work areas that can run partly in parallel.

At a glance

#	Workstream	Focus
1	**Dataset and preprocessing**	Corpus, queries, relevance labels, text normalization
2	**Classical retrieval**	TF-IDF, BM25, query-likelihood
3	**Semantic retrieval**	Embedding-based retrieval, hybrid fusion
4	**Evaluation and integration**	Metrics, comparisons, failure examples, demo

Workstreams

WORKSTREAM 1

Dataset and preprocessing

Load the corpus, understand the queries and relevance labels, prepare the document data, and test normalization, stemming and lemmatization.

WORKSTREAM 2

Classical retrieval

Implement TF-IDF, BM25 and query-likelihood retrieval, using the course notes and tutorial code.

WORKSTREAM 3

Semantic retrieval

Study the advanced notebook, implement embedding-based retrieval, and investigate hybrid fusion.

WORKSTREAM 4

Evaluation and integration

Validate the metrics, compare rankings and query times, prepare result tables and failure examples, and integrate the system into a usable demo.